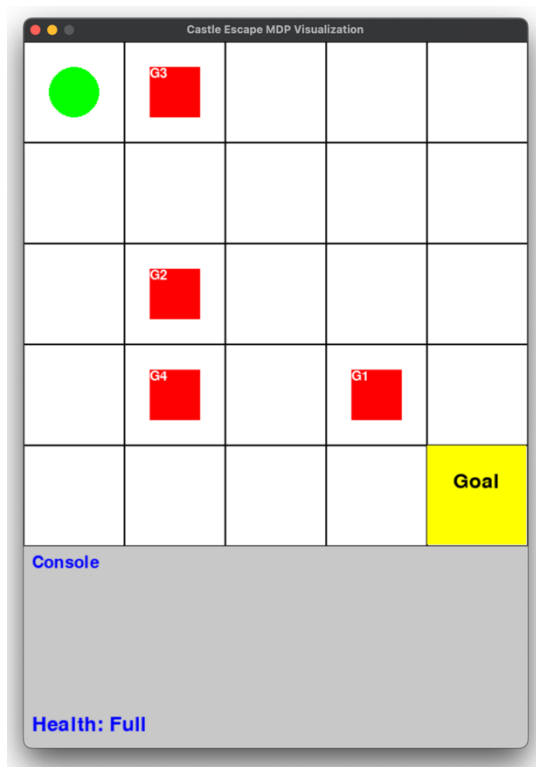


Homework 2: Reinforcement Learning

Due: Monday, March 10 at 1:35 PM

- Start early and ask for help if needed! This assignment has multiple parts, and your code may take a long time to run.
- You may discuss this assignment with your classmates. However, all discussion should be kept at a conceptual level, and code must not be shared. Violation of this policy is considered a violation of the course's academic integrity policy.
- Submitted code **must** be your own work. Any assignment that contains code that is not your own (including code from classmates, AI, or other external sources) will automatically receive a 0.
- If you use any sources other than the course notes, course readings, or official Python documentation, please cite them. A comment at the top of your code is sufficient.
- Submit your assignment on Gradescope. Please read the submissions section for instructions about what files are needed. Please make sure to upload all required files and double check that the autograder can run your code!

The Problem



In this homework, you will be using reinforcement learning techniques to build an agent that can play the Escape the Castle game. In this game, an agent tries to “escape the castle” by reaching the goal state in the corner of the game board, while also trying to evade or fight hidden guards.

The game is always played on a 5x5 grid, as shown above. Cells are labeled in a zero-indexed manner so that the upper left corner is (0, 0) and the bottom right is (4, 4). The player always begins in (0, 0), and the goal state is always in (4, 4). Additionally, there are four guards in the grid, which may be in any cells other than the start and goal positions. The guards' positions are randomized at the start of the game and are not known to the player. Guards' positions do not change during the game.

States

The player's current state can be represented by three factors:

- **Player position:** The cell that player is currently occupying
- **Player health:** The player has three possible health states: full, injured, and critical. The player's health always begins at full and is reduced when they lose a fight. If the player's health reaches the critical state, they lose the game.
- **Guard presence:** The player can detect whether or not a guard is in their cell, and if so, which guard (G1, G2, G3, or G4).

Since there are 25 possible player positions, 3 possible health states, and 5 possible guard options (including the "no guard" option), there are 375 possible states. Both MFMC.py and MBMC.py provide hash functions to map these states onto an integer between 0 and 374.

Actions

A player's legal actions depend on whether they are in a cell with a guard. If there is no guard, a player can move up, down, left, or right. 90% of the time, the player will move in their desired direction. 10% of the time, the player will "slip" and move in a random undesired direction. In either case, the player can never take an action that would cause them to leave the grid.

If a player is in a cell with a guard, they may not move. Instead, they must either hide or fight. A player's chances of succeeding in these actions are determined by the guard's keenness (for hiding) or strength (for fighting). A given guard's strength and keenness are not known to the player at the start of training, but they will be the same across all episodes.

If a player takes the hide action, they randomly succeed or fail, with probabilities based on the guard's keenness. If the player succeeds, they move to a random adjacent cell. If they fail, they instead are forced to fight the guard.

If a player takes the fight action (or fails their hide action), they fight the guard and randomly succeed or fail based on the guard's strength. If the player succeeds, they move to a random adjacent cell. If the player fails, they also move to a random adjacent cell, but they also lose health, possibly ending the game.

Rewards

The player receives a large positive reward for reaching their goal, a small positive reward for winning a fight, and a negative reward for losing a fight. Specifically, the rewards are as follows:

- Reaching the Goal: +10,000 reward for successfully reaching the goal at grid (4,4).
- Winning a Fight: +10 reward for defeating a guard in combat.
- Losing a Fight: -1,000 penalty for losing a fight and suffering health damage.
- Defeat: -1,000 penalty if the player loses the game

The Code

The `mdp.py` file defines the underlying MDP and provides interfacing methods through which your player can interact with the game environment. No changes should be made to this file (though you may want to read through it). The `vis.py` file defines the PyGame environment that visualizes gameplay—doing so may help visually catch issues with your implemented algorithms. You will need to have the `pygame` and `gymnasium` packages installed to run this code.

You **must** use the `env.step()` function to interact with the environment. Variables listed under the constructor (`def __init__(self):` in `mdp_gym.py`) may be accessed, but not modified directly from within your code. No other functions except `env.step()`, `env.reset()`, and `env.action_space.sample()` defined in `mdp_gym.py` or `vis_gym.py` may be called from within your code. If you modify or interact with the environment in other ways, you will lose points, and (depending on the extent), you may receive a 0 on the assignment.

The two files you will be updating are `MBMC.py` and `MFMC.py`. Instructions about your tasks are provided below *and* in the comments of these Python files.

Your Tasks

For this assignment, you will be implementing two reinforcement learning approaches: model-based Monte Carlo and Q-learning.

Model-Based Monte Carlo

It is possible to solve this problem using MBMC by determining the transition model and solving an MDP. However, with a state space of this size, doing so would be very time consuming! Instead, we will simply focus on estimating the probable outcomes of fight actions.

In the `MBMC.py` file, complete the `estimate_victory_probability()` function, which executes **random** actions in the environment until the specified number of episodes is completed, and aggregates the data. Next, the function computes the probabilities of defeating each of the four guards when a fight action is used, based on the data gathered during training episodes. The function returns a NumPy array containing four probabilities, one associated with defeating each

guard under the FIGHT action. For more information, read through the comments provided in MBMC.py.

Q-Learning

Now, we will fully train our agent to play the game using Q-learning. To do so, you will be writing the `Q_learning` function in `MFMC.py`. Again, you will be training your agent by playing through many episodes of the game. However, this time, you will be using an epsilon-greedy strategy and updating a Q-table as you go.

Simulate n episodes, using the `env.step()` function to interact with the environment. Calling `env.step()` at any cell returns a new observation, the reward, a Boolean indicating whether you are at a terminal state, and some miscellaneous info about the environment.

During training, use the epsilon-greedy approach to choose actions. This means with probability epsilon, you should choose a random legal action. With probability $1 - \text{epsilon}$, instead the optimal action: the action a that maximizes $Q(s, a)$. At the end of the **episode**, epsilon is decreased by multiplying it by a decay rate. The starting value and decay rate for epsilon are provided in the code (though you may want to experiment with other values for the decay rate).

During training, we will maintain a Q-table containing values of $Q(s, a)$ for all state-action pairs as a dictionary, as specified in the code. At the start of training, all values in the Q-table are 0. Since Q-learning is a temporal difference algorithm, we will update our Q values immediately after each action using the equation discussed in class:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s, a, s') + \gamma * \max_{a'} Q(s', a') - Q(s, a))$$

To account for the fact that different states will get explored different numbers of times during training, we will use a different value of α for each state-action pair and we will lower these α values over time. In particular, define:

$$\alpha = 1 / (1 + \text{\#of previous updates to } Q(s, a))$$

This means that the first time $Q(s, a)$ is updated, α will equal 1, and α will then decrease as more updates occur. In order to keep track of this, you will want a matrix of shape (num. states, num. actions), initialized to zeros, and updated such that executing `num_updates[s, a]` gives you the number of times $Q(s, a)$ has been updated. Using this matrix, you can calculate α directly before each update.

The provided code will automatically save your final Q-table to a pickle file, which you will need to upload with your final submission.

Submissions and Grading

The MFMC.py automatically saves the updated Q-table at the end of training to a file named Q_table.pickle. Please upload this file, along with all code files (MFMC.py, MBMC.py, vis_gym.py, mdp_gym.py) directly to Gradescope (no need to zip files, please make sure you do not upload a directory). Please make sure the filenames match these exactly.

This assignment will be graded using a mix of automatic and manual grading. For the MBMC algorithm, the autograder will check whether your probabilities are close to the true values. For Q-learning, the autograder will assign points based on the rewards earned by your final trained agent over several episodes of training. Partial credit will be possible for both. Assignments that try to hardcode solutions or otherwise manipulate the autograder will receive a 0.

Manual grading will be used to evaluate how well you implement how well you implement the MBMC and Q-learning algorithms, including properly using the gymnasium environment, properly updating your Q-table, and properly using the epsilon-greedy strategy. You may gain (or lose) points in manual grading, regardless of how well your code performs in the autograder.